



SOUTHEAST UNIVERSITY
Meeting the Challenges of Time

Department of Computer Science and Engineering Final assignment

Course Title: Artificial Intelligence

Course Title: Artificial Intelligence

Course code: CSE364

Section: 4

Assignment Version: Final Assignment

Prepared for:

Dr. Mijanur Rahman

Associate Professor,

Department of Computer Science and Engineering,
Southeast University.

Submitted By:

Md Rasel Hosen

ID: 2022000000008

Submission date: 19 June, 2024

CONTENTS

Multi-layer perceptron

- Introduction to MLPs
- Architecture of MLPs
- Why we choose MLP over the SLP
- Why do we choose sigmoid activation function over the other activation function
- Mathematical operation of MLP (Hand Written)
- Weight initialization techniques
- Forward Propagation
- Backward Propagation
- Learning goals
- Application of Multilayer perceptron

Introduction to Multi-Layer Perceptron

In the realm of machine learning, neural networks have become a pivotal tool for solving complex problems. Among the various types of neural networks, the Multi-Layer Perceptron (MLP) stands out as one of the foundational architectures. An MLP is a class of feedforward artificial neural network (ANN) that consists of at least three layers of nodes: an input layer, one or more hidden layers, and an output layer. This assignment delves into the structure, functioning, applications, and advantages of MLPs.

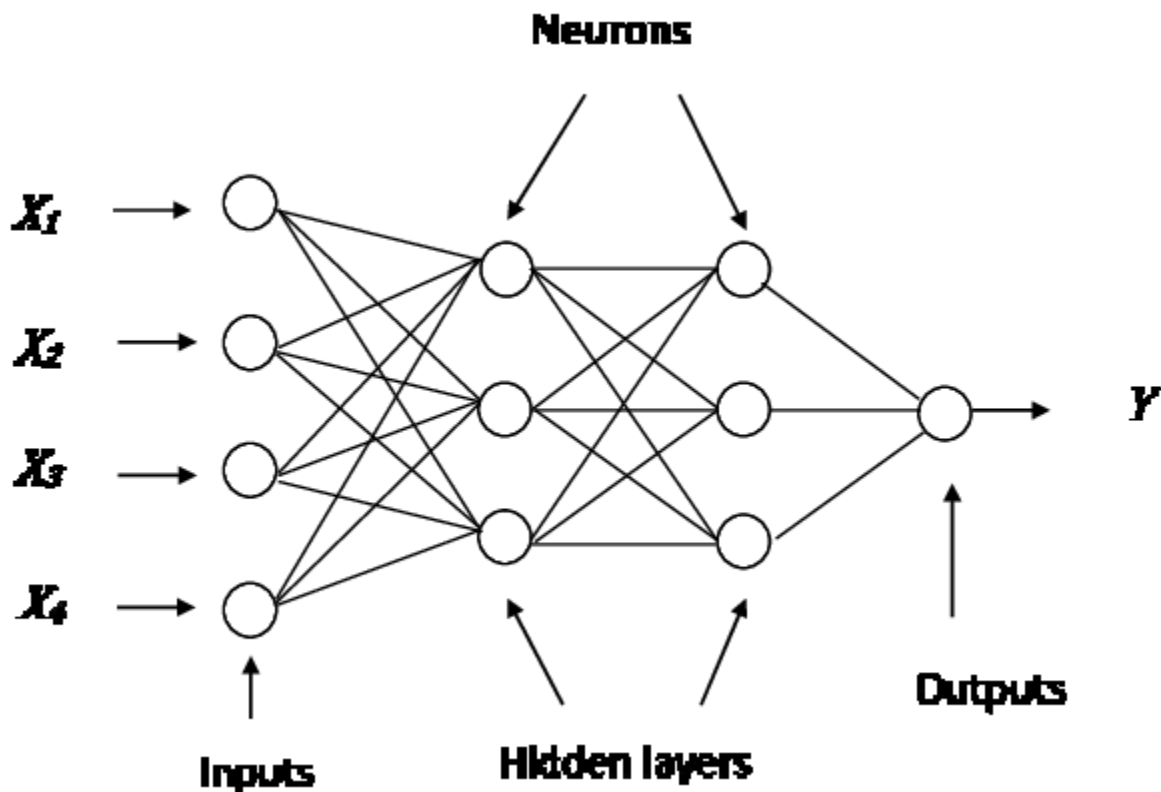


Fig: Multi-layer perceptron

Architecture of MLP

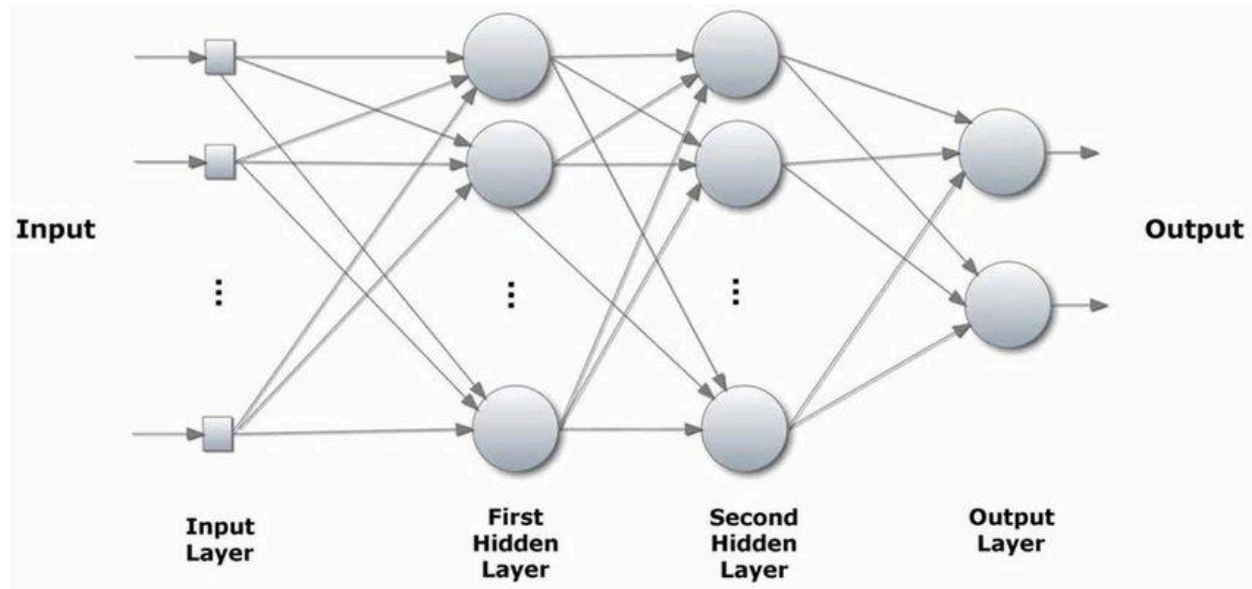


Fig: Architecture of Multilayer perceptron

1. Input Layer

- **Function:** Receives the input data.
- **Structure:** Contains neurons equal to the number of input features.

2. Hidden Layers

- **Function:** Perform computations and extract features from the input data.
- **Structure:** One or more layers, each containing a specified number of neurons.
- **Activation Function:** Typically uses non-linear activation functions such as ReLU (Rectified Linear Unit), Sigmoid, or Tanh to introduce non-linearity into the network.

3. Output Layer

- **Function:** Produces the final output of the network.
- **Structure:** Contains neurons equal to the number of output classes (for classification problems) or one neuron (for regression problems).
- **Activation Function:** Depends on the task; for example, Softmax is commonly used for multi-class classification, Sigmoid for binary classification, and no activation (linear) for regression.

Why we choose Multi-layer perceptron over the single layer perceptron

Multi-Layer Perceptrons (MLPs) are preferred over Single Layer Perceptrons (SLPs) for several reasons, primarily related to their ability to model complex, non-linear relationships in data. Here are the key reasons:

1. Capability to Model Non-Linear Relationships

- SLPs: A single-layer perceptron can only model linearly separable data. It uses a linear activation function (e.g., step function) and can only classify data that can be separated by a straight line (in 2D) or a hyperplane (in higher dimensions).
- MLPs: With one or more hidden layers and non-linear activation functions (such as ReLU, Sigmoid, or Tanh), MLPs can model complex, non-linear relationships. This makes them capable of handling data that is not linearly separable, which is common in real-world scenarios.

2. Expressive Power and Universal Approximation

- SLPs: Limited in expressive power as they can only represent linear decision boundaries.
- MLPs: According to the Universal Approximation Theorem, MLPs with at least one hidden layer and a non-linear activation function can approximate any continuous function to any desired degree of accuracy, given sufficient neurons in the hidden layer. This means MLPs can theoretically represent any decision boundary or function.

3. Feature Representation and Hierarchical Learning

- SLPs: Do not perform feature extraction; they simply apply a linear transformation to the input features.
- MLPs: The hidden layers in MLPs enable hierarchical learning, where each successive layer can learn increasingly abstract and complex features from the raw input data. For instance, in image recognition, early layers might detect edges, while deeper layers could detect shapes and objects.

4. Handling More Complex Problems

- SLPs: Suitable for simple classification tasks where the classes are linearly separable.

- MLPs: Can handle more complex tasks such as multi-class classification, regression, and even sequence prediction when combined with other techniques (like Recurrent Neural Networks).

5. Improved Performance and Flexibility

- SLPs: Generally less powerful and flexible, resulting in poorer performance on complex datasets.
- MLPs: Offer better performance and flexibility. By adjusting the number of hidden layers and neurons, MLPs can be tailored to the specific complexity of the problem at hand.

Practical Example:

Consider a problem of classifying points in a 2D space:

- SLP Example: If the points are arranged in a manner where a straight line can separate the classes, a single layer perceptron can solve this problem effectively.
- MLP Example: If the points are arranged in a circular pattern, where one class is inside the circle and another class is outside, a single layer perceptron fails as the data is not linearly separable. An MLP, with its ability to learn non-linear decision boundaries, can model this complex relationship by learning the circular boundary.

Why we choose sigmoid activation function over the other activation function in multi-layer perceptron

Why Sigmoid Here, the output is between zero and one. Logic gates output is either zero or one. The sigmoid function always outputs zero to one. So, we need a function which will give output from zero to one / between zero to one. And so, we are

using sigmoid function now.

1. Output Interpretation

- Probability Output: The Sigmoid function outputs values in the range (0, 1), which can be interpreted as probabilities. This is particularly useful in binary classification tasks where the output represents the probability of a sample belonging to a particular class.

- **Logistic Regression:** In models where logistic regression is a component, the Sigmoid function naturally fits since it forms the core of logistic regression, mapping the input to a probability.

2. Smooth Gradient

- **Non-linearity and Differentiability:** The Sigmoid function is smooth and differentiable, which is essential for gradient-based optimization techniques like backpropagation. This allows for the calculation of gradients required for updating weights during training.
- **Chain Rule:** The smooth gradient helps in applying the chain rule in backpropagation without sudden jumps or discontinuities, aiding stable training.

3. Compatibility with Certain Network Architectures

- **Restricted Boltzmann Machines (RBMs) and Autoencoders:** Sigmoid functions are particularly useful in the hidden layers of certain types of networks like RBMs and autoencoders, where binary-like outputs are desirable.

Limitations and Comparisons with Other Activation Functions

While the Sigmoid function has its advantages, it also has limitations compared to other activation functions like ReLU (Rectified Linear Unit) and Tanh:

1. Vanishing Gradient Problem

- **Issue:** The Sigmoid function can cause the vanishing gradient problem during backpropagation, particularly for deep networks. Gradients can become very small for inputs far from zero, leading to slow convergence or even stalling during training.
- **ReLU Advantage:** ReLU does not suffer from the vanishing gradient problem as severely because it maintains gradients for positive input values.

2. Output Range

- **Sigmoid:** The output range is $(0, 1)$, which can limit the range of activations and slow down learning as the output can get saturated.
- **Tanh:** The Tanh function has an output range of $(-1, 1)$, which centers the data and often results in faster convergence in practice.

3. Computational Efficiency

- Sigmoid: Computationally more expensive due to the exponential function involved.
- ReLU: Much simpler and faster to compute as it only involves a threshold operation.

When to Use Sigmoid

Despite its limitations, the Sigmoid function is still useful and chosen in specific scenarios:

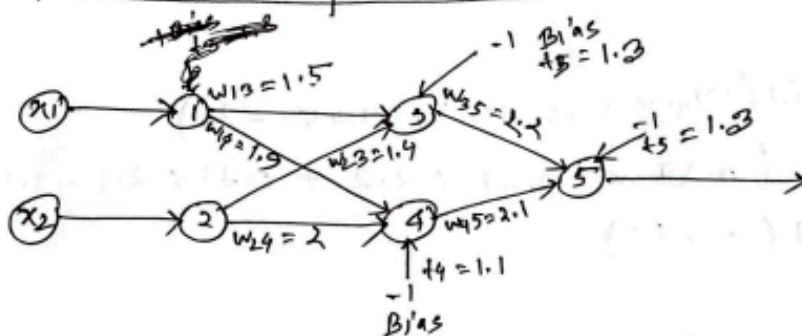
- Binary Classification: In the output layer of binary classification networks.
- Certain Hidden Layers: When the desired output needs to be in the (0, 1) range or when using architectures that require Sigmoid activation for compatibility.
- Legacy Systems: When working with or maintaining older neural network models that were originally designed with Sigmoid activations.

Mathematical operation on Multi-layer perceptron

Mathematical operation of MLP

input		output
A	B	A XOR B
0	0	1
0	1	0
1	0	0
1	1	1

Fig = Three layer network for solving Exclusive-NOR operation.



Epoch-1

Iteration 1

$$x_1 = 0 \quad x_2 = 0$$

$$\begin{aligned}
 y_3 &= \text{sigmoid} (x_1 \times w_{13} + x_2 \times w_{23} - t_3) \\
 &= \text{sigmoid} (0 \times 1.5 + 0 \times 1.4 - 1.3) \\
 &= \text{sigmoid} (-1.3) \\
 &= \frac{1}{1 + e^{(-1.3)}} \\
 &= 0.21
 \end{aligned}$$

$$\begin{aligned}
 y_4 &= \text{sigmoid}(x_1 \times w_{14} + x_2 \times w_{24} - t_4) \\
 &= \text{sigmoid}(0 \times 1.9 + 0 \times 2 - 1.1) \\
 &= \text{sigmoid}(-2.9) \\
 &= \frac{1}{1 + e^{-2.9}} \\
 &= 0.71
 \end{aligned}$$

$$\begin{aligned}
 y_5 &= \text{sigmoid}(y_3 \times w_{35} + y_4 \times w_{45} - t_5) \\
 &= \text{sigmoid}(\cancel{0.21} \times 0.22 \times 2.2 + 0.71 \times 2.1 - 1.1) \\
 &= \text{sigmoid}(0.853) \\
 &= \frac{1}{1 + e^{-0.853}} \\
 &= 0.70
 \end{aligned}$$

Error Calculation -

$$\begin{aligned}
 \text{err}_1 &= 1 - 0.70 \\
 &= 0.30
 \end{aligned}$$

Now Calculating Error Gradients

$$\begin{aligned}
 e_{g5} &= y_5 * (1 - y_5) * \text{err}_1 \\
 &= 0.70 \times (1 - 0.70) \times 0.30 \\
 &= 0.147
 \end{aligned}$$

$$\begin{aligned}
 e_{g3} &= y_3 \times (1 - y_3) \times e_{g5} \times w_{35} \\
 &= 0.21 \times (1 - 0.21) \times 0.147 \times 2.2 \\
 &= 0.053
 \end{aligned}$$

$$\begin{aligned}
 e_{g4} &= y_4 \times (1 - y_4) \times e_{g5} \times w_{45} \\
 &= 0.71 \times (1 - 0.71) \times 0.147 \times 2.1 \\
 &= 0.063
 \end{aligned}$$

Then starting corrected weights and thresholds by using the following form. (we considered learning rate $\alpha = 0.1$)

$$e_{g5} =$$

$$w_{35} += \alpha \times y_3 \times e_{g5}$$

$$w_{35} = w_{35} + \alpha \times y_3 \times e_{g5}$$

$$= 2.2 + 0.1 \times 2.1 \times 0.147$$

$$= 2.23$$

$$w_{45} += \alpha \times y_4 \times e_{g5}$$

$$w_{45} = w_{45} + \alpha \times y_4 \times e_{g5}$$

$$= 2.1 + 0.1 \times 0.71 \times 0.147$$

$$= 2.11$$

$$t_5 += \alpha \times (-1) \times e_{g5}$$

$$t_5 = t_5 + \alpha \times (-1) \times e_{g5}$$

$$= 1.3 + 0.1 \times (-1) \times 0.147$$

$$= 1.28$$

$$w_{13} + = \alpha \times x_1 \times e_{g3}$$

$$\begin{aligned} w_{13} &= w_{13} + \alpha \times x_1 \times e_{g3} \\ &= 1.5 + 0.1 \times 0 \times 0.053 \\ &= 1.5 \end{aligned}$$

$$\begin{aligned} w_{23} &= w_{23} + \alpha \times x_2 \times e_{g3} \\ &= 1.4 + 0.1 \times 0 \times 0.053 \\ &= 1.4 \end{aligned}$$

$$\begin{aligned} w_{14} &= w_{14} + \alpha \times x_1 \times e_{g4} \\ &= 1.9 + 0.1 \times 0 \times 0.063 \\ &= 1.9 \end{aligned}$$

$$\begin{aligned} w_{24} &= w_{24} + \alpha \times x_2 \times e_{g4} \\ &= 2 + 0.1 \times 0 \times 0.063 \\ &= 2 \end{aligned}$$

$$\begin{aligned} b_4 &= b_4 + \alpha \times (-1) \times e_{g4} \\ &= 1.1 + 0.1 \times (-1) \times 0.063 \\ &= 1.09 \end{aligned}$$

New weights are

$$w_{13} = 1.5$$

$$w_{23} = 1.4$$

$$w_{14} = 1.9$$

$$w_{24} = 2$$

So on...

Weight initialization techniques

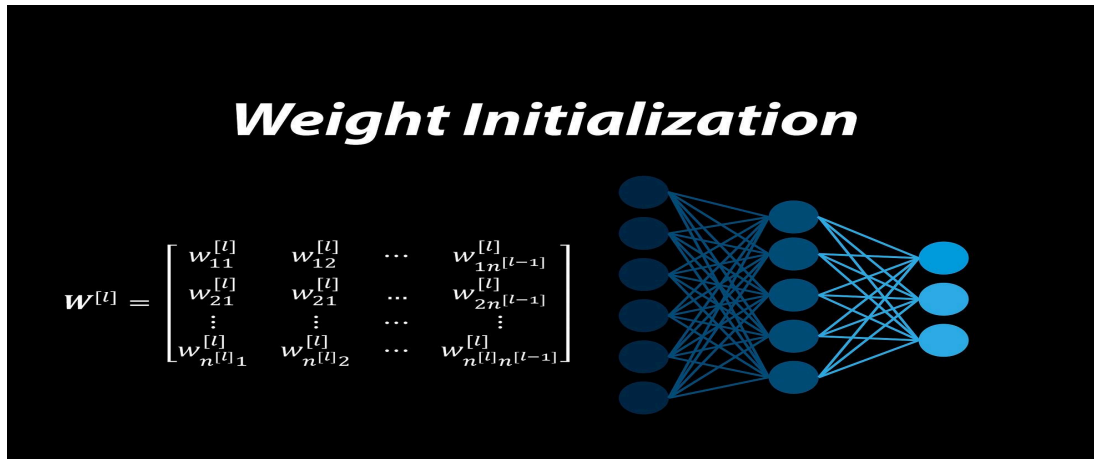


Fig: Weight Initialization

Zero Initialization:

- **Description:** All weights are initialized to zero.
- **Problems:** This approach is generally not used because it breaks the symmetry between neurons in the same layer. If all weights are zero, all neurons will compute the same output and gradients, making it impossible for the network to learn.

Random Initialization:

- **Description:** Weights are initialized randomly, typically using a uniform or normal distribution.
- **Benefits:** Helps to break the symmetry, allowing different neurons to learn different features.
- **Problems:** If not scaled properly, random weights can lead to vanishing or exploding gradients, especially in deep networks.

Practical Considerations

1. Bias Initialization:

- Biases are typically initialized to zero or small positive values. Initializing biases to zero is common practice and usually does not affect the symmetry-breaking property of weight initialization.
2. **Layer-Specific Initialization:**
 - Different layers might benefit from different initialization schemes. For example, convolutional layers might use He initialization while fully connected layers use Xavier initialization, depending on the activation functions used.
 3. **Normalization:**
 - Techniques like Batch Normalization can help mitigate issues related to poor weight initialization by normalizing activations throughout training.

Forward propagation in Multi-layer perceptron

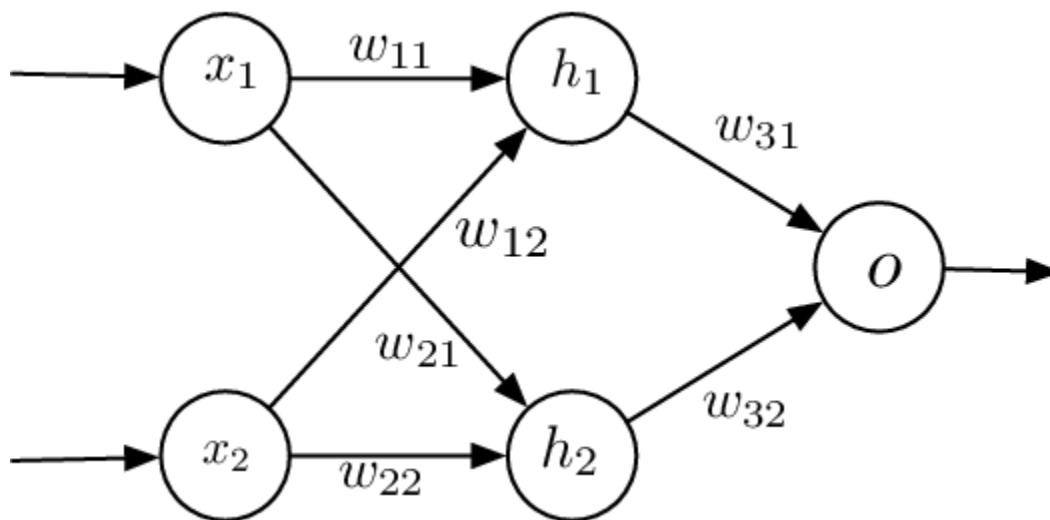


Fig: Forward propagation

Forward propagation is the process of computing the output of a neural network for a given input by propagating the input data through the network layer by layer until the output layer is reached. The output of each layer serves as the input to the next layer, and computations are performed sequentially according to the network's architecture.

1. **Input Layer:**
 - The input data is fed into the input layer of the neural network.
2. **Hidden Layers:**
 - For each hidden layer, the weighted sum of inputs is computed for each neuron.

- The weighted sum is then passed through an activation function to introduce nonlinearity.
 - The output of the activation function becomes the input to the next layer.
3. Output Layer:
- Similar to the hidden layers, the output layer computes the weighted sum of inputs for each neuron.
 - The weighted sum is passed through an activation function appropriate for the task (e.g., softmax for classification, linear for regression).
4. Output Calculation:
- The output of the output layer is the final prediction or classification made by the neural network for the given input.

Backward propagation

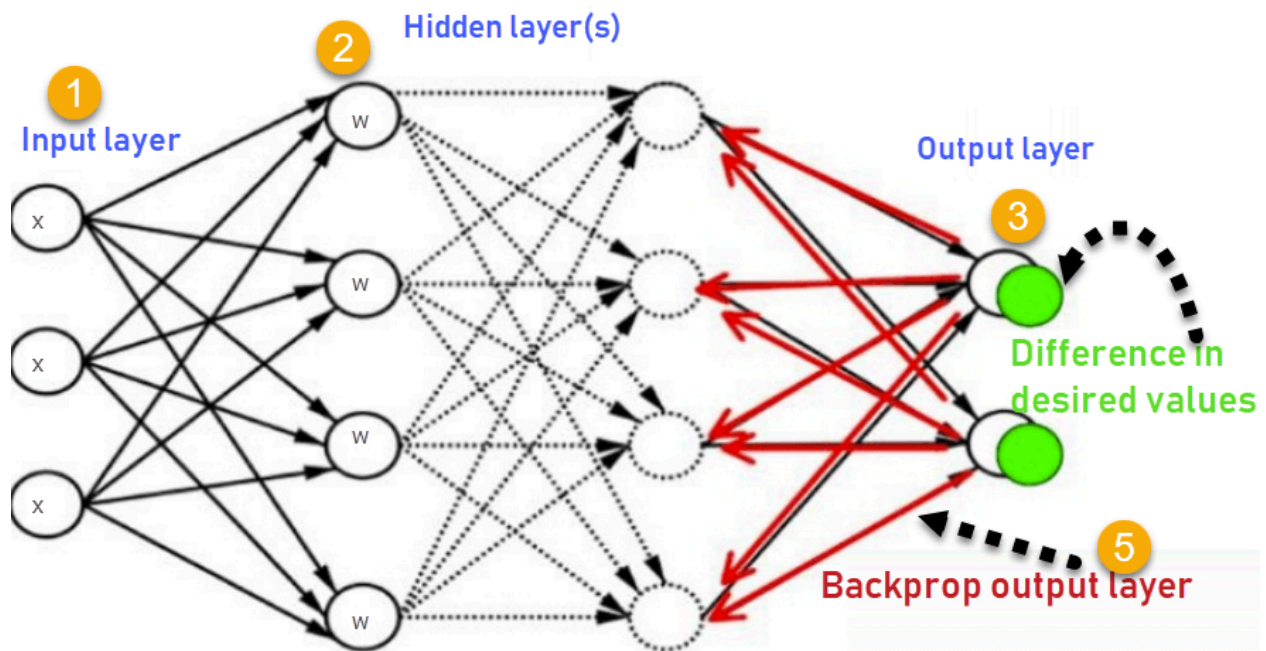


Fig:Backpropagation Training

Backpropagation is the process of computing the gradients of the loss function with respect to the weights of the network, which is essential for updating the weights during training to minimize the loss. It involves two main steps: computing the gradients of the loss function with respect to the network's outputs (backward pass) and using these gradients to compute the gradients of the loss function with respect to the network's weights (gradient descent).

1. Backward Pass:
 - Compute the error or loss between the predicted output and the true output using a suitable loss function (e.g., mean squared error for regression, cross-entropy loss for classification).
 - Propagate the error backward through the network to compute the gradients of the loss function with respect to the output of each neuron.
2. Gradient Descent:
 - Use the chain rule of calculus to compute the gradients of the loss function with respect to the weights of the network.
 - Update the weights of the network in the direction that minimizes the loss function using an optimization algorithm such as stochastic gradient descent (SGD) or its variants.
 - The learning rate determines the size of the update step taken during optimization.
3. Iterative Optimization:
 - Repeat the forward propagation and backpropagation steps for multiple iterations (epochs) until the model converges to a satisfactory solution or a stopping criterion is met.
 - Monitor the training/validation performance to prevent overfitting and adjust hyperparameters as needed.

Learning Goals

- Know the basic terminology for neural nets
- Given the weights and biases for a neural net, be able to compute its output from its input
- Be able to hand-design the weights of a neural net to represent functions like XOR
- Understand how a hard threshold can be approximated with a soft threshold
- Understand why shallow neural nets are universal, and why this isn't necessarily very interesting

Source:

https://www.cs.toronto.edu/~rgrosse/courses/csc321_2018/readings/L05%20Multilayer%20Perceptrons.pdf

Application of Multi-layer perceptron

1. Handwritten Digit Recognition

- **Application:** Optical Character Recognition (OCR)

- **Description:** MLPs are used in systems like the MNIST digit recognition, where the goal is to classify images of handwritten digits (0-9).
- **Process:** The input to the MLP is a flattened vector representing the pixel values of the image. The MLP processes these inputs through multiple hidden layers to extract features and ultimately classify the digit in the output layer.
- **Impact:** Widely used in digitizing handwritten documents and postal code recognition in mail sorting.

2. Spam Email Detection

- **Application:** Email Filtering
- **Description:** MLPs can classify emails as spam or not spam based on features extracted from the email content, such as the presence of certain keywords, frequency of terms, and other metadata.
- **Process:** The email content is preprocessed and converted into a numerical representation (e.g., TF-IDF vectors). The MLP then processes these vectors through its layers to classify the email.
- **Impact:** Enhances email security and user experience by filtering out unwanted emails.

3. Financial Forecasting

- **Application:** Stock Market Prediction
- **Description:** MLPs are used to predict stock prices and other financial metrics based on historical data.
- **Process:** Historical stock prices, trading volumes, and other relevant financial indicators are fed into the MLP. The network learns patterns and relationships to make predictions about future prices.
- **Impact:** Assists traders and financial analysts in making informed investment decisions.

4. Medical Diagnosis

- **Application:** Disease Prediction
- **Description:** MLPs can assist in diagnosing diseases by analyzing medical data, such as patient records, test results, and symptoms.
- **Process:** Patient data is input into the MLP, which processes the information through its layers to identify patterns indicative of specific diseases.
- **Impact:** Supports healthcare professionals in early detection and diagnosis of diseases, leading to better patient outcomes.

5. Customer Churn Prediction

- **Application:** Business Analytics
- **Description:** MLPs are used by companies to predict whether customers are likely to stop using their service or product.
- **Process:** Customer data, including usage patterns, purchase history, and demographic information, is input into the MLP. The network predicts the likelihood of churn based on learned patterns.
- **Impact:** Enables businesses to implement targeted retention strategies and improve customer loyalty.

6. Image Classification

- **Application:** Object Recognition in Images
- **Description:** MLPs are employed in systems that need to classify objects within images, such as identifying types of animals, vehicles, or other objects.
- **Process:** Images are converted into pixel intensity vectors, which are then fed into the MLP. The network processes these vectors to classify the objects.
- **Impact:** Used in various fields such as autonomous vehicles, security systems, and content moderation.

7. Natural Language Processing (NLP)

- **Application:** Sentiment Analysis
- **Description:** MLPs analyze text to determine the sentiment expressed, such as positive, negative, or neutral.
- **Process:** Text is tokenized and transformed into numerical vectors (e.g., word embeddings). The MLP processes these vectors to classify the sentiment.
- **Impact:** Used in social media monitoring, customer feedback analysis, and market research.